

Name : Meghavarshini M
Class : III AIML B
Reg. No : RA2211026050054

LEXICAL ANALYZER

EX. NO. 1

AIM: To write a program to implement a lexical analyzer.

ALGORITHM:

1. Start.
2. Get the input C++ source code filename from the user.
3. Open the file and read its content character by character.
4. Identify and classify each token as a keyword, identifier, number, operator, or symbol.
5. If a word is an identifier, assign it a unique number and store it in a symbol table.
6. Generate tokens for each lexeme:
 - a. Identifier → <id, number>
 - b. Operator → <op, operator>
 - c. Number → <const, value>
 - d. Keyword → <keyword>
7. Display the stream of tokens on the console.
8. Close the file.
9. Stop.

PROGRAM:

```
#include <iostream>
```

```
#include <fstream>
```

```

#include <cctype>

#include <unordered_set>

using namespace std;


unordered_set<string> keywords = {"int", "float", "double", "char", "if", "else", "while",
"for", "return", "void", "main"};

unordered_set<char> operators = {'+', '-', '*', '/', '=', '<', '>', '!', '&', '|'};

unordered_set<char> symbols = {'(', ')', '{', '}', '[', ']', ';', ':'};


bool isKeyword(const string &word) {

    return keywords.find(word) != keywords.end();

}


bool isOperator(char ch) {

    return operators.find(ch) != operators.end();

}


bool isSymbol(char ch) {

    return symbols.find(ch) != symbols.end();

}


bool isNumber(const string &word) {

    for (char ch : word) {

        if (!isdigit(ch)) return false;
    }
}

```

```
    }  
  
    return true;  
}
```

```
void lexicalAnalyze(const string &filename) {  
  
    ifstream file(filename);  
  
    if (!file) {  
  
        cerr << "Error opening file!" << endl;  
  
        return;  
    }  
  
    string word;  
  
    char ch;  
  
    while (file.get(ch)) {  
  
        if (isalpha(ch)) {  
  
            word += ch;  
  
            while (file.get(ch) && (isalnum(ch) || ch == '_')) {  
  
                word += ch;  
  
            }  
  
            file.unget();  
  
            if (isKeyword(word)) {  
  
                cout << "Keyword: " << word << endl;  
  
            } else {  
  
                cout << "Identifier: " << word << endl;  
  
            }  
        }  
    }  
}
```

```

        word.clear();
    } else if (isdigit(ch)) {
        word += ch;

        while (file.get(ch) && isdigit(ch)) {
            word += ch;
        }

        file.unget();

        cout << "Number: " << word << endl;

        word.clear();
    } else if (isOperator(ch)) {
        cout << "Operator: " << ch << endl;
    } else if (isSymbol(ch)) {
        cout << "Symbol: " << ch << endl;
    }
}

file.close();
}

int main() {
    string filename;

    cout << "Enter the C++ source code filename: ";

    cin >> filename;

    lexicalAnalyze(filename);

    return 0;
}

```

INPUT(C++ FILE):

```
#include <iostream>
```

```
using namespace std;
```

```
int main(){
```

```
    int a = 9;
```

```
    int b = 10;
```

```
    cout<<a+b;
```

```
    return 0;
```

```
}
```

OUTPUT:

Enter the C++ source code filename: add.cpp

Identifier: include

Operator: <

Identifier: iostream

Operator: >

Identifier: using

Identifier: namespace

Identifier: std

Symbol: ;

Keyword: int

Keyword: main

Symbol: (

Symbol:)

Symbol: {

Keyword: int

Identifier: a

Operator: =

Number: 9

Symbol: ;

Keyword: int

Identifier: b

Operator: =

Number: 10

Symbol: ;

Identifier: cout

Operator: <

Operator: <

Identifier: a

Operator: +

Identifier: b

Symbol: ;

Keyword: return

Number: 0

Symbol: ;

Symbol: }